

CP353002 · Section 04

PrintFlow

ระบบจัดการร้านรับพิมพ์เอกสารออนไลน์

Spring Boot · Layered Architecture · Design Patterns

ณัฐชา (P1) · ปฏิภาณ (P2) · รัชชานนท์ (P3) · อาณัฐ (P4)

ใบสั่งงาน

ตัวอย่างการคิดราคาริจจากระบบ

เอกสารสี 40 หน้า × 2 400.00

เย็บมุม × 2 4.00

โค้ด SAVE10 ลด 10% -40.40

ยอดสุทธิ **363.60**



ที่มาและวัตถุประสงค์

คิดราคาด้วยมือ

หลายประเภทงาน + คำนวณส่วนลด คิด
ผิดง่าย

โทรถามสถานะ

ลูกค้าไม่รู้ว่าจะงานถึงไหน

ไม่มีตัวเลขสรุป

ร้านไม่รู้ยอดขาย ไม่รู้ว่าอะไรขายดี

เป้าหมายของ PrintFlow

- ลูกค้าสั่งพิมพ์ออนไลน์ รู้ราคาก่อนส่งงาน และติดตามสถานะได้
- พนักงานและผู้ดูแล จัดการงาน การชำระเงิน โปรโมชัน และดูรายงานยอดขาย
- ประยุกต์ Layered Architecture, SOLID, Design Pattern พร้อม test และ CI/CD

ผู้ใช้ 3 บทบาท และทีม 4 คน

ลูกค้า

สิ่งพิมพ์ ติดตามสถานะ ยกเลิกตอนรอยืนยัน

พนักงาน

ยืนยันงาน เปลี่ยนสถานะ บันทึกการชำระเงิน

ผู้ดูแล

บริการ โปรโมชัน บัญชีผู้ใช้ รายงานยอดขาย

P1 ภาครัฐฯ

Auth, Security, Admin, Report, DevOps

P2 ปฏิภาณ

บริการ, โปรโมชัน, Strategy, ฐานข้อมูล

P3 รัชชานนท์

การสั่งงาน, Chain of Responsibility, Test

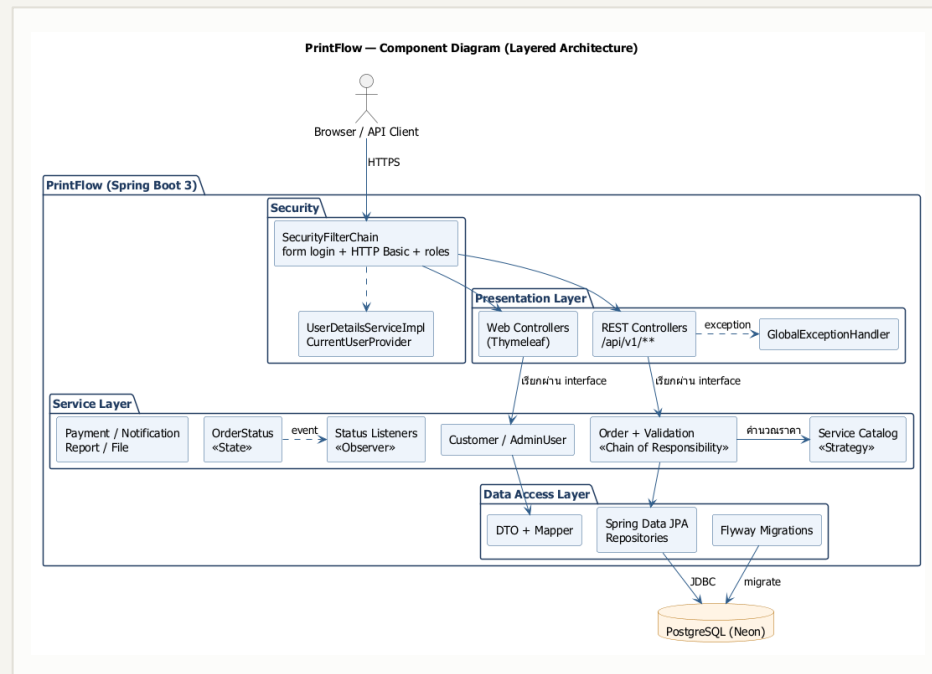
P4 อาณัฐ

สถานะงาน, State, Observer, การชำระเงิน

14 use case ครบทั้ง 3 บทบาท

PrintFlow · CP353002 Sec 04

Layered Architecture 4 ชั้น



Presentation

REST API /api/v1 และหน้าเว็บ Thymeleaf

Service

interface + impl และ pattern ทั้ง 4 ตัว

Repository

Spring Data JPA

Database

PostgreSQL บน Neon + Flyway V1-V9

Security และ REST API

43

REST endpoint บน Swagger UI

3

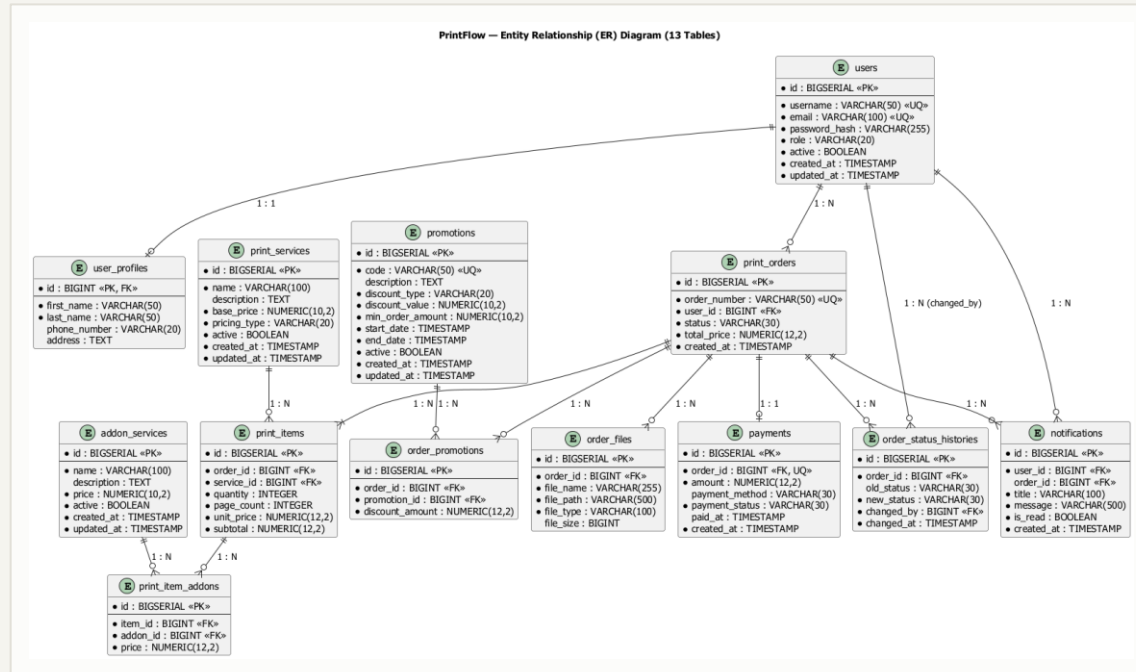
role: CUSTOMER STAFF ADMIN

403

ดู order ของคนอื่นไม่ได้

- กฎสิทธิ์รวมที่ SecurityConfig และเช็คเจ้าของข้อมูลใน service อีกชั้น
- รหัสผ่านเข้ารหัส BCrypt · หน้าเว็บมี CSRF · API ใช้ HTTP Basic
- Error ทุกตัวตอบ JSON รูปแบบเดียวผ่าน @RestControllerAdvice

ฐานข้อมูล 13 ตาราง



13

ตารางใน PostgreSQL

V1-V9

Flyway ไม่แก้ไฟล์เก่า

1:1 · 1:N · M:N

ความสัมพันธ์ครบทุกแบบ

Strategy: คิดราคาแบบไม่มี if-else

ประเภทงาน	สูตร (คลาส Strategy)
ขาวดำ	ราคา × หน้า × ชุด
สี	เกิน 50 หน้า ลด 10%
ภาพถ่าย	ราคา × จำนวนแผ่น
งานเก็บเล่ม	ราคา × จำนวนชุด
ส่วนลด	Percentage / FixedAmount

เอกสารสี 40 หน้า × 2 + เย็บมุม + SAVE10

5.00 × 40 × 2 400.00

2.00 × 2 4.00

ลด 10% -40.40

ยอดสุทธิ

363.60

Open/Closed

เพิ่มประเภทงานใหม่ = เพิ่มคลาส + ค่าใน enum ไม่แก้ Service หรือ Strategy เดิม

Chain of Responsibility: ตรวจ 5 ชั้น



ไม่ผ่านชั้นไหน หยุดทันที

ตอบ 400 พร้อมเหตุผล ยังไม่มีอะไรลงฐานข้อมูล

เพิ่มกฎใหม่ = เพิ่ม handler

ต่อเข้าสายใน OrderValidationChainConfig

Order 1 ใบ มีหลาย Item

PrintOrder.java · ฝั่ง inverse

```
@OneToMany(mappedBy = "order",  
            fetch = FetchType.LAZY)  
private List<PrintItem> items;
```

PrintItem.java · ฝั่งเจ้าของ FK

```
@ManyToOne(fetch = FetchType.LAZY)  
@JoinColumn(name = "order_id")  
private PrintOrder order;
```

LAZY

โหลด item เฉพาะตอนใช้

ไม่ cascade ลบ

order เป็นหลักฐานการเงิน

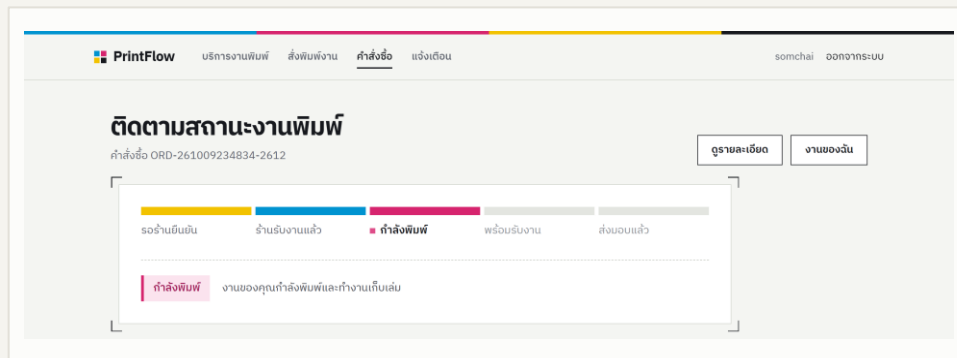
POST ตอบ 201

/api/v1/orders = 201 Created

index 3 ตัว

user_id, status, created_at

State: แต่ละสถานะรู้กฎของตัวเอง



รอร้านยืนยัน

ร้านรับงานแล้ว

กำลังพิมพ์

พร้อมรับงาน

ส่งมอบแล้ว

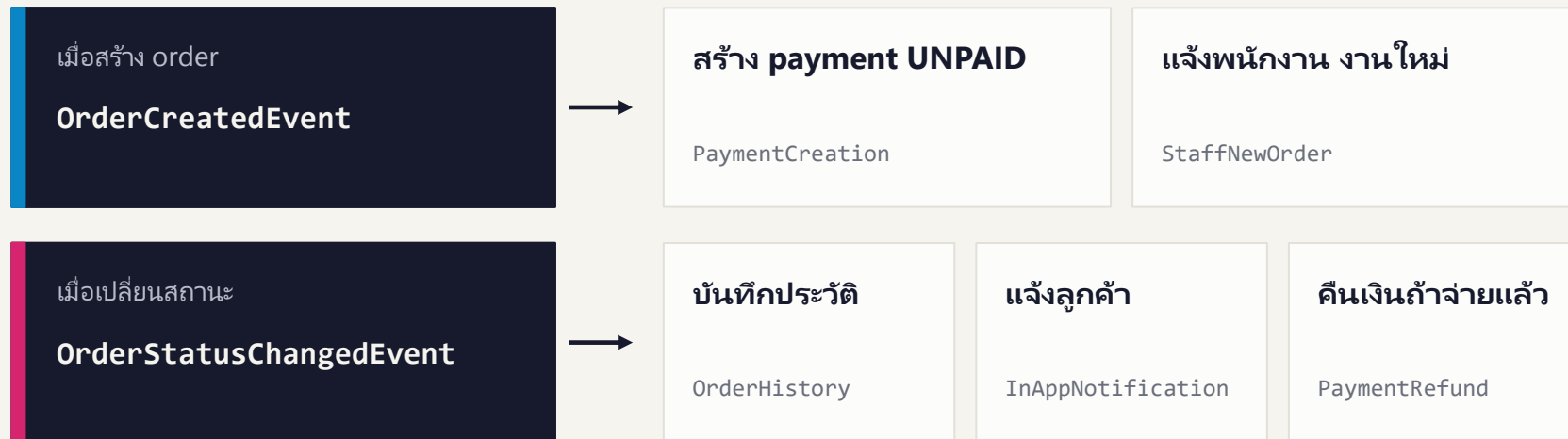
ยกเลิกแล้ว

AbstractOrderState

ทุก action ปฏิเสธเป็นค่าเริ่มต้น
สถานะไหนทำได้ค่อย override

- เปลี่ยนผิวดลำดับ ตอบ 409 ไม่บันทึก
- เมฆพนักงานถาม State ว่าไปต่อไหนได้
- ลูกค้ายกเลิกได้ตอนรอยืนยันเท่านั้น

Observer: event 2 ตัว listener 5 ตัว

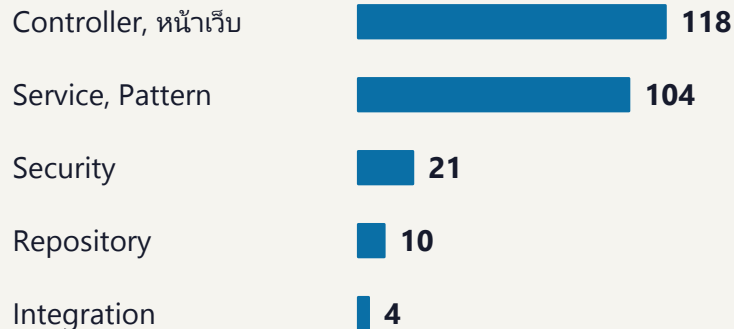


การชำระเงิน จ่ายซ้ำ หรือจ่ายงานที่ยกเลิก ตอบ 409 · ยกเลิกหลังจ่าย ระบบเปลี่ยนเป็น REFUNDED

Testing และ DevOps

257

test ผ่าน 100%



Git Flow

branch ต่อคน → develop → main ผ่าน PR

GitHub Actions

build และรัน test ทุก push

Docker

multi-stage, JRE 17, เวลาไทย

Render + Neon

deploy อัตโนมัติ ค่าลบอยู่ใน env

สรุปผล: ครบตามวัตถุประสงค์

วัตถุประสงค์	สิ่งที่ทำได้	ผล
สั่งออนไลน์ รู้ราคา ติดตามสถานะ	คิดราคาทันที + แจ้งเตือนทุกครั้ง	ครบ
ร้านจัดการงาน เงิน รายงาน	14 use case ครบ 3 บทบาท	ครบ
Layered, SOLID, Pattern	4 ชั้น + Strategy State Observer Chain	ครบ
Test และ CI/CD	257 test ผ่าน, Actions → Render	ครบ

สิ่งที่ได้เรียนรู้

Pattern ทำให้เพิ่มความสามารถได้โดยไม่แก้ของเดิม เช่น กฎจำกัดสิทธิ์โค้ดต่อคน เพิ่มเป็น handler ใหม่ในสาย

ลำดับการสาธิต · 3 นาที

Demo และ Q&A

printflow-ogm4.onrender.com

Swagger UI: </swagger-ui.html>

ขอบคุณครับ

ชั้นที่ 1 · P3

ลูกค้าสั่งพิมพ์ ระบบคิดราคาให้

ชั้นที่ 2 · P4

พนักงานยืนยันงาน บันทึกชำระเงิน

ชั้นที่ 3 · P4

ลูกค้าเห็นแจ้งเตือนและสถานะใหม่

ชั้นที่ 4 · P1

Admin ดูรายงาน และ Swagger